Refactor While Green



It's tempting to start factoring out duplicate code while you're still writing your specs. Avoid that temptation: get your specs passing first, and *then* refactor. That way, you can use your specs to check your refactoring.

Refactoring

Both test cases have identical expressions setting up the `ledger` test double. You can remove the duplication by moving these into a common `before` hook. Place the following code just inside the first `context` block:

05-unit-specs/05/expense_tracker/spec/unit/app/api_spec.rb

```
let(:expense) { { 'some' => 'data' } }

before do
  allow(ledger).to receive(:record)
    .with(expense)
    .and_return(RecordResult.new(true, 417, nil))
end
```

Now, remove the setup code from both examples:

05-unit-specs/05/expense_tracker/spec/unit/app/api_spec.rb

```
it 'returns the expense id' do
  post '/expenses', JSON.generate(expense)

  parsed = JSON.parse(last_response.body)
  expect(parsed).to include('expense_id' => 417)
end

it 'responds with a 200 (OK)' do
  post '/expenses', JSON.generate(expense)
  expect(last_response.status).to eq(200)
end
```

These refactored specs report "just the facts" of the expected behavior. When a POST request arrives, the body should contain the expense ID returned by our ledger and the response code should be 200.

The examples are a little more DRY now ("Don't Repeat Yourself," an approach explained in the book *The Pragmatic Programmer [HT00]*). It may be tempting to DRY them up even further by also moving the `post '/expenses' ...` lines into the `before` hook. Such a move would be overkill, though.

Putting the `post` in the `before` hook will likely get in the way of future specs for this context. For instance, adding support for XML clients would require a different HTTP header. If you send the HTTP request from your setup code, you won't be able to tweak the headers for just your XML examples.